



UNIVERSITÀ
DEGLI STUDI
DI BRESCIA

DIPARTIMENTO DI Ingegneria dell'Informazione

Corso di Laurea in Ingegneria delle Tecnologie per l'Impresa Digitale

Relazione Finale

Monitoraggio, manutenzione e sicurezza per smart city: dalla gestione manuale ad un'architettura automatizzata basata su Zabbix

Relatore:

Chiar.mo Prof. Francesco Gringoli

Laureando :

Brando Calderara

Matricola n. 746352

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare presso l'azienda Secoval Srl e durante il successivo approfondimento in sede di stesura dell'elaborato dal laureando Brando Calderara. Il progetto ha avuto come obiettivo principale la realizzazione di un sistema di monitoraggio centralizzato e automatizzato per le infrastrutture smart city del territorio della Comunità Montana di Valle Sabbia, superando i limiti dei precedenti sistemi.

Per l'implementazione del sistema è stata adottata la piattaforma open-source Zabbix che è stata estesa tramite l'integrazione con script locali. In primo luogo il sistema fornisce la capacità di discovery dei device, il popolamento dei loro attributi con informazioni attinenti e l'importante funzione di monitoraggio della presenza attiva sulla rete di ognuno di essi. In secondo luogo, per le telecamere di videosorveglianza, è stato sviluppato un sistema asincrono per la ricezione di allarmi relativi a manomissioni volontarie o involontarie.

Infine per garantire l'integrazione con i processi e i flussi aziendali già presenti il sistema è stato integrato con la piattaforma di ticketing aziendale, in modo che ogni intervento di manutenzione possa essere tracciato nello stesso paradigma usato in precedenza dagli operatori di Secoval.

Il risultato del progetto è un'infrastruttura funzionante, già rilasciata in ambiente di produzione, che permette di avere una visione unificata dello stato delle infrastrutture smart city del territorio e dei tempi di risposta ai disservizi di rete significativamente ridotti.

Indice

Sommario	3
Elenco delle figure	6
Elenco delle tabelle	7
Listings	8
1 Introduzione	9
1.1 L'azienda: Secoval S.r.l.	9
1.2 Il contesto dello stage e della tesi	10
1.3 Obiettivi e analisi dei requisiti	10
1.3.1 Classificazione dei requisiti	11
2 Tecnologie utilizzate	13
2.1 Ubuntu Server	13
2.2 Zabbix 7.0 LTS	14
2.3 Python, Venv e i package pip utilizzati	15
2.4 Interacta	16
3 Architettura e configurazione	17
3.1 Setup della macchina virtuale Ubuntu e gestione dei dati	17
3.1.1 Setup della VM Ubuntu	17
3.2 Zabbix nella funzione di monitoraggio della smart city	17
3.2.1 Architettura generale dell'infrastruttura	17
3.2.2 L'autodiscovery degli host da parte di Zabbix	18
3.3 I fogli di calcolo aziendali	19
3.3.1 Struttura e ruolo dei due file Excel aziendali	19
3.3.2 L'estrazione dei dati	20
3.4 Implementazione dell'aggiornamento degli host	20
3.4.1 Associazione dei metadati agli host	21
3.4.2 Memorizzazione dei messaggi di log	21
3.5 Georeferenziazione e GIS integrato in Zabbix	22
3.6 Reportistica e KPI	23
3.6.1 Esportazione dati evento tramite custom AlertScript	23
3.6.2 Salvataggio locale in CSV	24
3.6.3 Elaborazione dati e report	24

4	Rilevamento manomissioni TVCC	26
4.1	Logiche di Polling vs Trapping in Zabbix	26
4.2	Descrizione dell'implementazione	26
4.3	Configurazioni tamperdetection e motiondetection hikvision tramite XML	27
4.4	Sviluppo del middleware e ascolto degli stream Hikvision ISAPI . . .	27
4.4.1	Implementazione del listener	27
4.4.2	Esecuzione continuativa e log	28
4.4.3	Trigger actions e integrazione con Interacta	28
5	Integrazione con il sistema di ticketing Interacta	30
5.1	Scopo e architettura dell'integrazione	30
5.1.1	Autenticazione Server-to-Server	31
5.2	Generazione e firma del token JWT	32
5.2.1	Struttura del Token e scelta dell'algoritmo	32
5.2.2	Implementazione in Python	32
5.3	Apertura o chiusura del ticket	33
6	Conclusioni	36
6.1	Raggiungimento degli obiettivi	36
6.2	Limiti del sistema e sviluppi futuri	37
6.3	Valutazione Personale	37
	Glossario	38
	Bibliografia	42
	Ringraziamenti	43

Elenco delle figure

1.1	Organigramma Secoval (Secoval S.r.l., 2026b)	9
1.2	Mappa relativa ai comuni che si avvalgono del sistema smart city	11
1.3	Schema a blocchi come astrazione del sistema	12
2.1	Logo Ubuntu	13
2.2	Logo Zabbix	14
2.3	Logo Python	15
2.4	Logo Interacta	16
3.1	GIS integrato in Zabbix	22
3.2	GIS integrato in Zabbix - dettaglio di un'area	23
5.1	Esempio di ticket in Interacta	31

Elenco delle tabelle

3.1	Tabella Excel degli asset(dati sensibili offuscati)	19
3.2	Tabella Excel delle coordinate geografiche(dati sensibili offuscati)	19
6.1	Riepilogo dei requisiti soddisfatti	36

Listings

3.1	Struttura della singola sottorete estratta da PRTG	18
3.2	Setup dell'esecuzione periodica in Crontab	21
3.3	Configurazione logrotate per un file di log	21
3.4	Custom bash script per l'esportazione in CSV	24
5.1	Snippet Python per la generazione e firma del JWT con algoritmo RS512	32
5.2	Snippet Python per la apertura o chiusura di un ticket	34

Capitolo 1

Introduzione

1.1 L'azienda: Secoval S.r.l.

Secoval srl è una società a capitale interamente pubblico, partecipata dalla Comunità Montana di Valle Sabbia e da oltre 40 enti locali. Opera come supporto tecnologico e operativo per le Pubbliche Amministrazioni del nord-est bresciano, con sede a Vestone. Il suo ruolo è supportare la trasformazione digitale dei Comuni, compresi quelli più periferici, fornendo servizi amministrativi, informatici e gestionali. (Secoval S.r.l., 2026a)

Le principali aree di operatività dell'azienda includono:

le infrastrutture IT e cloud per la gestione di data center, servizi cloud, sicurezza informatica e interoperabilità per gli enti pubblici,

i sistemi informativi territoriali che comprendono lo sviluppo di reti (inclusa la fibra ottica), la digitalizzazione del catasto e la cartografia,

i servizi sovracomunali e gestionali per la gestione dei tributi locali ed il supporto amministrativo e al personale degli enti.

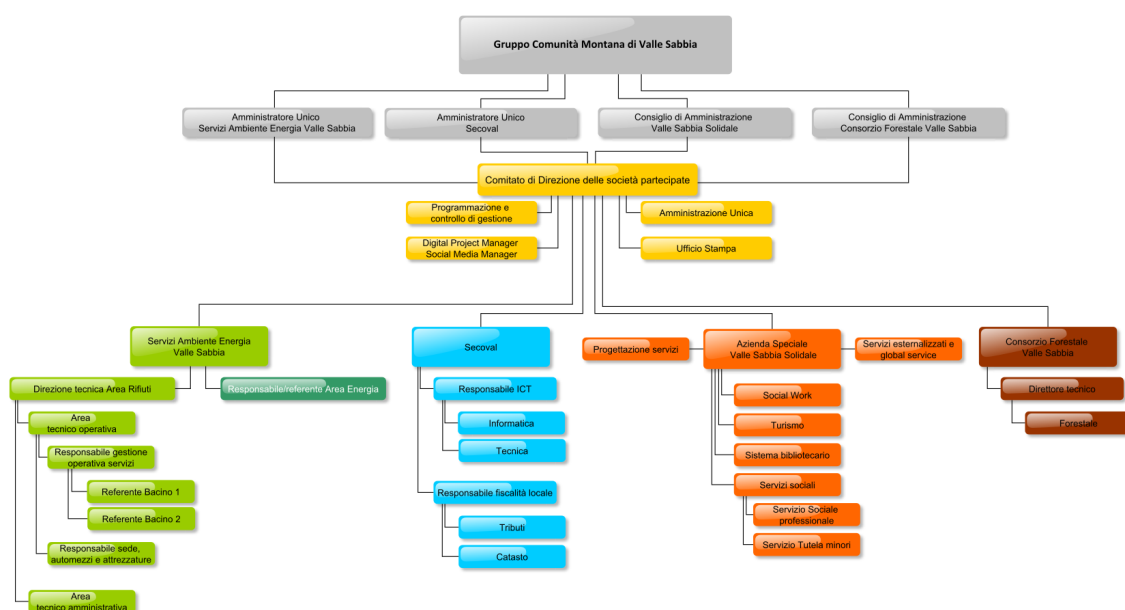


Figura 1.1: Organigramma Secoval (Secoval S.r.l., 2026b)

Nel contesto tecnologico delle smart city, Secoval gestisce apparati tra cui telecamere per la videosorveglianza urbana (TVCC), Access Point (AP) per la connettività pubblica, switch industriali e gateway LTE per le aree non cablate. Essi fanno parte di reti distribuite sul territorio per l'erogazione di servizi diretti ai cittadini.

Il progetto svolto durante il tirocinio si inserisce direttamente in questo scenario operativo, con l'obiettivo di superare la gestione manuale implementando un'architettura automatizzata e centralizzata basata su Zabbix, in modo che si realizzi un sistema di monitoraggio solido per la rete di apparati di Secoval, abbastanza vasta e appena installata quindi instabile. Il lavoro ha previsto l'automazione del discovery e dell'assegnazione dei metadati degli apparati tramite fogli Excel, l'integrazione con i flussi di allarme nativi dei dispositivi periferici (come gli eventi ISAPI Hikvision) e la comunicazione sincrona e crittografata con la piattaforma cloud di ticketing aziendale Interacta per la gestione asincrona dei guasti da parte degli operatori.

1.2 Il contesto dello stage e della tesi

Il progetto di stage ha avuto come obiettivo la progettazione e l'implementazione di un'architettura di monitoraggio centralizzata basata su Zabbix 7.0. Esso è destinato alla gestione di una delle infrastrutture di rete per smart city che Secoval gestisce. L'esigenza primaria era migliorare e superare i limiti dei sistemi di monitoraggio precedenti, come PRTG (Paessler Router Traffic Grapher), software proprietario con un sostanziale costo per la gestione di reti di certe dimensioni (Paessler, 2026) e con un'elevata gestione manuale nella implementazione precedente al tirocinio.

Il successivo lavoro di tesi ha permesso di studiare l'astrazione dell'architettura, studiare l'autenticazione con JWT verso la piattaforma di ticketing aziendale e sviluppare gli accorgimenti sul codice necessari per la messa in produzione.

L'infrastruttura monitorata è eterogenea e distribuita sul territorio. I dispositivi principali comprendono e si dividono in:

- **Telecamere di videosorveglianza (TVCC):** principalmente telecamere di marca Hikvision, che richiedono il controllo di raggiungibilità e dell'integrità del flusso video.
- **Access Point (AP):** nodi per l'erogazione di connettività pubblica.
- **Switch:** switch industriali, e non, per l'aggregazione del traffico periferico.
- **Dispositivi LTE:** dispositivi delle precedenti 3 tipologie che si basano sull'utilizzo di gateway per l'uplink in aree remote non coperte da fibra ottica.

1.3 Obiettivi e analisi dei requisiti

Il progetto formativo ha mirato a trasformare l'infrastruttura di rete (videosorveglianza, Wi-Fi pubblico) che serve 31 Comuni in un sistema smart city monitorabile e facilmente scalabile, lavorando su tre pilastri:

Sviluppo e miglioramento del monitoraggio di rete.

Integrazione degli apparati (telecamere, switch, AP) con la Cartografia GIS per creare uno strumento di gestione georeferenziato.

Integrazione con la piattaforma cloud di ticketing aziendale Interacta, garantendo la sicurezza delle comunicazioni machine-to-machine.

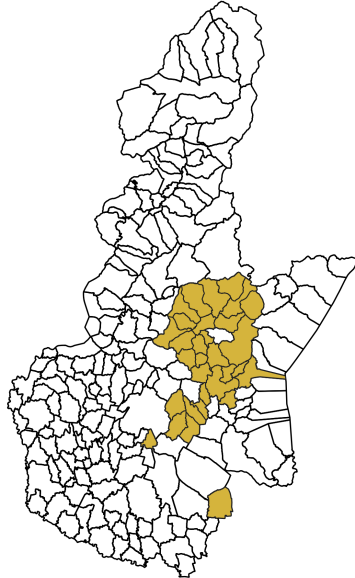


Figura 1.2: Mappa relativa ai comuni che si avvalgono del sistema smart city

1.3.1 Classificazione dei requisiti

L'analisi del contesto applicativo e dei vincoli imposti dall'infrastruttura e dall'azienda ha portato alla definizione dei seguenti requisiti tecnici e funzionali.

Requisiti:

Associazione metadati: attraverso un Configuration Management Database in formato Excel, il sistema deve associare agli host, identificati con gli indirizzi IP, metadati specifici (ID Host, ID Palo).

Gestione degli eventi TVCC: il monitoraggio deve intercettare logiche di allarme provenienti dalle telecamere, come il *Video Tampering* e il *Motion Detection*, mediante l'ascolto continuo degli stream ISAPI e visualizzarli nel frontend di Zabbix.

Automazione del ticketing (Interacta): al verificarsi di un disservizio o al suo rientro, il sistema deve interfacciarsi con le API REST del cloud aziendale Interacta per gestire l'apertura e la risoluzione automatica dei ticket. La comunicazione machine-to-machine deve supportare qualche tipo di autenticazione.

Elaborazione KPI: il sistema deve estrarre informazioni e metriche riguardo gli host, per offrire anche una visione d'insieme sullo stato del sistema.

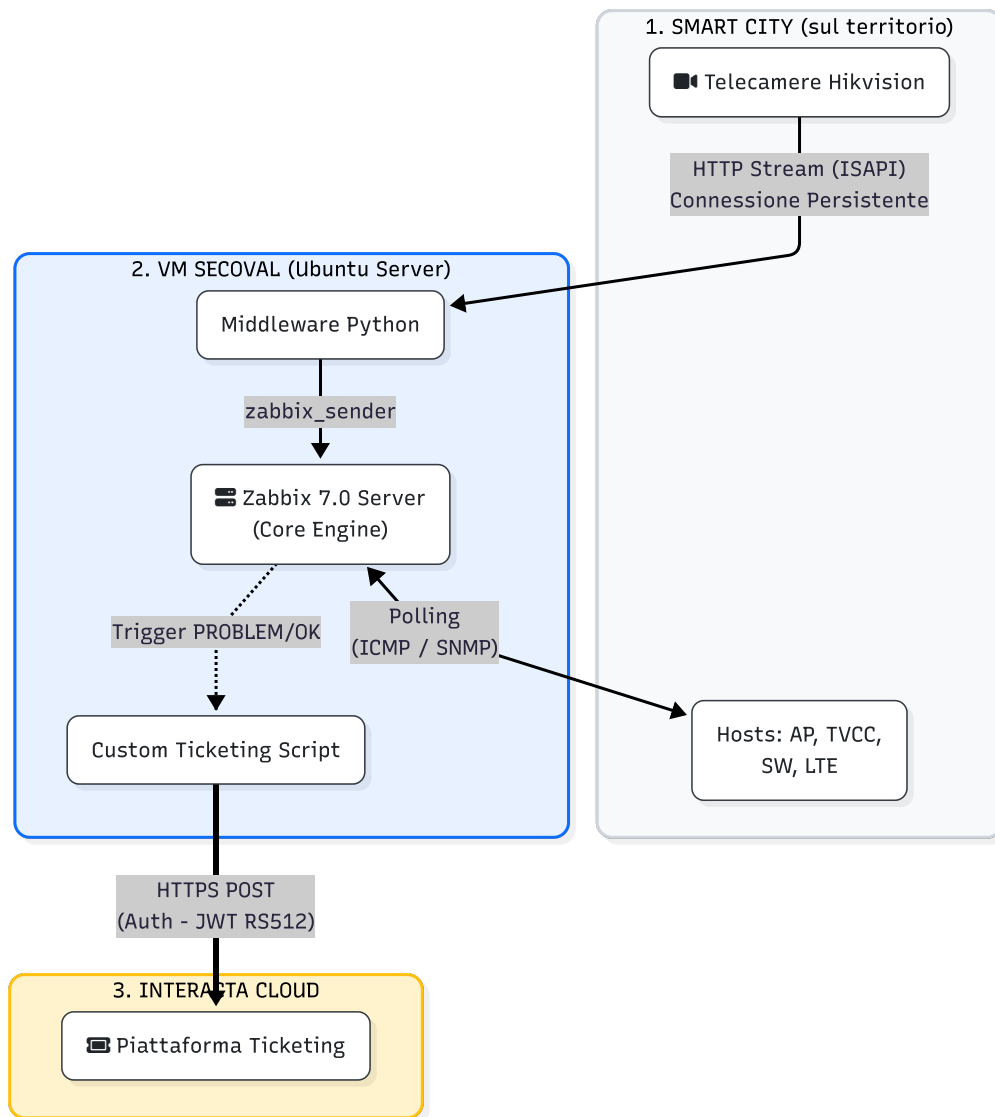


Figura 1.3: Schema a blocchi come astrazione del sistema

Capitolo 2

Tecnologie utilizzate

2.1 Ubuntu Server



Figura 2.1: Logo Ubuntu

L'infrastruttura di monitoraggio è stata messa in produzione su una macchina virtuale con sistema operativo Ubuntu Server(Canonical Ltd., 2026). Il sistema operativo mette a disposizione strumenti nativi utili per automatizzare il lavoro in background, rendendo il sistema più robusto e autonomo senza la necessità di appoggiarsi a software di terze parti.

Per orchestrare i flussi di dati e i vari script, il progetto ha sfruttato intensivamente tre componenti e tool di Ubuntu. Il primo è **Crontab** (Linux man-pages project, 2026a), lo schedulatore di Linux che permette di eseguire comandi a orari o intervalli prestabiliti. È stato utilizzato per gestire le operazioni periodiche. Ad esempio, la generazione e l'invio del report delle statistiche via email prima dell'inizio del turno aziendale.

Il secondo strumento impiegato è **Systemd**, ovvero il programma che gestisce i servizi del sistema operativo. Per garantire che lo script Python in ascolto sulle telecamere fosse sempre in funzione, è stato trasformato in un servizio di sistema (demone). Systemd lo avvia automaticamente all'accensione del server e lo riavvia in totale autonomia a seguito di eventuali errori imprevisi o cadute di rete o spegnimenti improvvisi.

Infine, è stata affrontata la gestione dello spazio su disco. Un sistema sempre attivo genera continuamente righe di log; tuttavia, se lasciate accumulare, questi file saturerebbero la memoria. Per prevenire questo rischio, è stato configurato **Logrotate**. Questa utility di sistema ogni settimana intercetta i file di log, li comprime per ridurne il peso e li archivia.

2.2 Zabbix 7.0 LTS



Figura 2.2: Logo Zabbix

Zabbix è una piattaforma open-source progettata per il monitoraggio, anche distribuito, e in tempo reale di metriche di rete, apparati, server e applicativi. L'architettura del sistema si fonda su un modello client-server altamente scalabile, composto dai seguenti moduli principali:

- **Zabbix server:** il motore centrale sviluppato in linguaggio C. Si occupa della raccolta dei dati, della valutazione dei *Trigger* (soglie di allarme) e delle *Actions*.
- **Database relazionale:** il livello di persistenza dei dati, tipicamente basato su PostgreSQL o MySQL (MySQL in questo progetto), in cui si archiviano le configurazioni e le serie temporali.
- **Web frontend:** l'interfaccia utente sviluppata in PHP, utilizzata per l'amministrazione del sistema, la configurazione visuale e la generazione di dashboard analitiche. *Apache2* è il web server impiegato in questo progetto.
- **Zabbix Proxy e Agent:** demoni impiegati sugli host periferici limitando il carico sul server centrale e dando la possibilità di distribuire il sistema. Non utilizzati in questo progetto.

Il funzionamento della piattaforma si basa su due metodi per l'acquisizione dei dati. Il primo è il *polling*, in cui il server interroga ciclicamente i dispositivi tramite protocolli standard come ICMP o SNMP. Il secondo è il *trapping*, che consente a sistemi esterni di spingere i dati verso il server Zabbix. Di questo secondo metodo si avvale l'utility a riga di comando **Zabbix Sender**.

Per questo progetto è stata adottata la versione 7.0 LTS (Long Term Support) (Zabbix LLC, 2024). La scelta di una release LTS garantisce stabilità. Utili dal punto di vista delle attività del tirocinio questa versione include:

- **API JSON-RPC:** l'API web di Zabbix, un'interfaccia strutturata che consente il controllo totale della piattaforma via codice. È stata utilizzata per popolare l'Host Inventory leggendo i metadati dalle configurazioni aziendali.
- **Custom AlertScripts:** la capacità di lanciare eseguibili esterni (Python, Bash...) al verificarsi di un evento. Questo modulo è stato impiegato per integrare Zabbix con la piattaforma cloud Interacta, e per creare un sistema di logging del sistema.

Per facilitare la lettura e la comprensione di questo elaborato si consiglia la lettura delle voci del glossario riguardanti Zabbix. Identificate da: "(Zabbix)".

2.3 Python, Venv e i package pip utilizzati

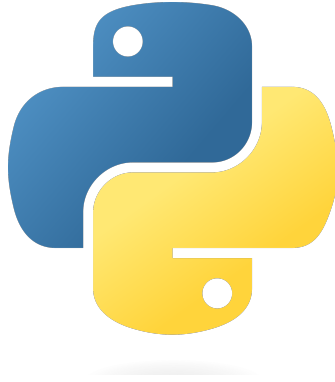


Figura 2.3: Logo Python

Python (Python Software Foundation, 2026b) è un linguaggio di programmazione ad alto livello, interpretato, tipizzato dinamicamente e orientato agli oggetti. Nel contesto di questo progetto, Python è stato il linguaggio principale utilizzato per estendere le funzionalità di Zabbix e interfacciare ulteriori sistemi.

L'esecuzione degli script all'interno del server Linux si è basata su **venv** (Virtual Environment). Esso è uno strumento nativo di Python e permette di creare ambienti virtuali isolati per ogni progetto. **Venv** si assicura che le librerie installate al suo interno non interferiscano con altre librerie di altri progetti o nel sistema operativo, aumentando la sicurezza e facilitando il porting del codice.

La gestione dei pacchetti e delle librerie è stata effettuata tramite **pip** (Package Installer for Python). Di seguito si riportano alcune librerie impiegate per lo sviluppo dei moduli del tirocinio (Python Software Foundation, 2026a):

- **pandas (e openpyxl)**: sono librerie molto diffuse per l'analisi e la gestione dei dati. Nel tirocinio sono state utilizzate per leggere e filtrare i file Excel aziendali contenenti l'elenco degli apparati. Grazie a queste librerie, il sistema è riuscito a estrarre dai file informazioni per configurare i dispositivi su Zabbix senza alcun inserimento manuale.
- **pyzabbix**: è una libreria creata appositamente per facilitare la comunicazione tra Python e Zabbix. Invece di dover costruire richieste HTTP in formato JSON-RPC, questa libreria fa da "traduttore" semplificando il codice. Nel progetto è stata il motore della configurazione dei dispositivi. Ha permesso agli script di autenticarsi sul server Zabbix tramite API token e di lanciare comandi per creare, aggiornare i dispositivi/host.
- **requests**: è una libreria progettata per inviare richieste HTTP. In questo progetto è servita per inviare i dati verso la piattaforma cloud Interacta.
- **PyJWT e cryptography**: sono librerie dedicate alla sicurezza informatica e alla crittografia. Sono state utilizzate per gestire l'autenticazione tra il server Linux e Interacta.

2.4 Interacta



Figura 2.4: Logo Interacta

Interacta è la piattaforma cloud aziendale adottata da Secoval per la gestione dei processi aziendali e del ticketing (Interacta, 2026). Nel contesto operativo del progetto, questo software rappresenta il punto di arrivo degli allarmi: quando un apparato sul territorio subisce un guasto o perde la connettività, gli operatori assegnati alla manutenzione ricevono una segnalazione sulla piattaforma Interacta, che potrà tradursi in un intervento di cui ogni step sarà tracciato secondo gli usi aziendali. Durante il tirocinio uno degli obiettivi è stato quello di realizzare l'integrazione, collegando Zabbix direttamente alle API REST di Interacta affinché i ticket di guasto (e i rispettivi rientri) vengano aperti e eventualmente risolti dall'infrastruttura software, rendendo più facile agli operatori la gestione dei guasti.

Capitolo 3

Architettura e configurazione

3.1 Setup della macchina virtuale Ubuntu e gestione dei dati

Per realizzare l'integrazione tra i dati Excel e il server Zabbix è stato necessario progettare un ambiente di esecuzione dedicato. L'architettura prevede l'impiego di una Macchina Virtuale (VM) basata su sistema operativo Ubuntu Linux, ospitata sull'infrastruttura di virtualizzazione aziendale (Nutanix). La scelta di Ubuntu Server garantisce stabilità, e compatibilità con software open-source come Zabbix.

3.1.1 Setup della VM Ubuntu

L'applicativo Zabbix viene eseguito su una virtual machine sui server Secoval, chiamata "SECZABBIX-TLC". Il sistema operativo utilizzato è Ubuntu Server (Ubuntu 24.04).

Nella VM a livello di sistema operativo, sono stati definiti contesti di esecuzione separati per necessità tecniche:

- l'utente **administrator**: impiegato per le fasi di sviluppo, l'utilizzo di SSH e per l'aggiornamento dei file Excel di input. La maggior parte della *codebase* risiede all'interno della sua directory di lavoro, specificamente nel percorso `/home/administrator/zabbixHostsScript/`.
- l'utente di sistema **zabbix**: utilizzato dal demone del server di monitoraggio per l'esecuzione in background degli script di alert e di integrazione.

3.2 Zabbix nella funzione di monitoraggio della smart city

3.2.1 Architettura generale dell'infrastruttura

L'infrastruttura che necessita di monitoraggio nel progetto di *smart city* è costituita da una serie di sottoreti virtuali *VLAN* che si sviluppano sul territorio. In esse sono collegati diversi host, di natura eterogenea, per oltre 1000 dispositivi attivi.

Come già discusso in precedenza, il monitoraggio delle sottoreti era affidato a Paessler PRTG Network Monitor. La migrazione a Zabbix è stata una scelta dettata

da fattori di costo e anche di automatizzazione: le API di Zabbix sono più facilmente utilizzabili e meglio documentate, per la loro natura open-source.

Per ogni comune si possono avere 4 sottoreti, per esempio nel comune di Lavenone:

TVCC-LAVE: ne fanno parte le telecamere principalmente di marca Hikvision, un gateway e un possibile server di gestione delle telecamere da parte del comune.

SW-LAVE: ne fanno parte gli switch, un gateway e un possibile server.

AP-LAVE: ne fanno parte gli access point e un gateway.

LTE-LAVE: sottorete facoltativa che contiene host di ogni tipo che si basano su tecnologia LTE o satellitare per comunicare da aree remote non coperte dai normali servizi.

La **funzionalità principale del sistema** consiste nell'applicare in Zabbix il template *ICMP ping* ad ogni host presente nelle VLAN del progetto. Questo template permette lanciare verso tutti gli host un comando *ping* ogni minuto e misurare, tra 3 diversi item, l'accessibilità dell'host sulla rete. In caso di comunicazione interrotta il relativo trigger lancia un problem con **Severity: high**.

3.2.2 L'autodiscovery degli host da parte di Zabbix

Essendo le sottoreti da monitorare già presenti nel sistema di monitoraggio precedente, PRTG, con il nome desiderato, è stato esportato un file *json* contenente un array di oggetti così strutturati:

```

1  {
2      "PRTG_ID": 4591,
3      "Full_Path": "smartCity/LAVENONE/TVCC-LAVE",
4      "GroupName": "TVCC-LAVE",
5      "IPv4_Subnet": "x.x.x.x/x"
6  },

```

Listing 3.1: Struttura della singola sottorete estratta da PRTG

Questo file è stato utilizzato per inizializzare in Zabbix le stesse sottoreti, attraverso uno script python.

In Zabbix gli host possono essere catalogati in *host groups*, a cui vengono assegnati da delle *actions* chiamate *discovery actions* dopo essere stati scoperti da delle *discovery rules*: sistemi automatici che trovano gli host presenti in una determinata sottorete.

I compiti svolti da questo script sono stati:

- Creazione di un nuovo *host group* per ogni "Full_Path"
- Creazione di una nuova *discovery rule* per ogni "Full_Path" contenente la sottorete IPv4 e la modalità con cui può scoprire nuovi host, in questo caso ICMP ping
- Creazione di un nuovo *discovery action* per ogni "Full_Path" che assegni l'host appena scoperto dalla discovery rule al suo host group e allo specifico template *ICMP ping*, citato in precedenza(3.2.1).

3.3 I fogli di calcolo aziendali

Una volta scoperti gli host e memorizzati nel database di Zabbix essi sono operativamente funzionanti, ma non contengono alcuna informazione che li riguarda. Molte delle loro caratteristiche, come nome e l’ID del palo a cui sono montati, non sono rilevabili dallo strumento software Zabbix perchè non sono stati memorizzati sui singoli host.

Per abilitare un’architettura di monitoraggio automatizzata, è stato necessario identificare una **sorgente della verità**, ovvero un’entità a cui il sistema potesse attingere per allineare il suo stato con lo stato fisico degli asset sul territorio.

Nel contesto della smart city gestita da Secoval Srl, il Source of Truth riguardo gli asset hardware e software che compongono l’infrastruttura della smart city e la configurazione e mappatura dei dispositivi dislocati sul territorio (telecamere IP, Access Point, gateway LTE, switch di rete) è rappresentato da 2 fogli di calcolo Excel: uno fondamentale come archivio e uno di supporto.

3.3.1 Struttura e ruolo dei due file Excel aziendali

Secoval, e le aziende che hanno implementato fisicamente la rete di telecamere sul territorio, gestiscono la conoscenza della propria infrastruttura smart city attraverso due file Excel principali:

1. **Excel degli asset:** questo documento contiene l’inventario di tutti gli asset hardware (come le telecamere, prevalentemente di marca Hikvision) installati nei vari comuni aderenti al progetto. Per ogni apparato sono mappati dati cruciali quali l’indirizzo IP statico, l’univoco ID Host aziendale, il costruttore, il modello e, dato fondamentale per la manutenzione fisica, l’ID Palo che identifica e da un nome alla posizione di installazione del dispositivo.

Nome Punto	ID Palo	Tipo Apparato	ID	ID Apparato	Marcia/Modello	Nome Host/DNS	Indirizzo IP	Netmask	Gateway	Server DNS 1	Server DNS 2	VLAN ID	Indirizzo MAC	Note
1_Sabbio_ITVCC	1_Sabbio_ITVCC	Telecamera di contesto	X	SABB-TVCC-01	HIKVISION ...	SABB-TVCC-01	X.X.X.X	255.255.255.X	X.X.X.X	X.X.X.X	X.X.X.X	XXX	XX:XX:XX:XX:XX:XX	
1_Sabbio_ITVCC	1_Sabbio_ITVCC	Switch Industriale	X	SABB-SW-TVCC-01	Switch 5 porte	SABB-SW-TVCC-01	X.X.X.X	255.255.255.X	X.X.X.X	X.X.X.X	X.X.X.X	XXX	XX:XX:XX:XX:XX:XX	

Tabella 3.1: Tabella Excel degli asset(dati sensibili offuscati)

2. **Excel delle coordinate geografiche:** Questo secondo file cataloga e associa gli ID Palo alle coordinate geografiche relative, espresse in latitudine e longitudine in gradi decimali.

Comune	Punto	Lotto	Nome Punto	WiFi Punto	Tipo Punto	Tecnologia	Indirizzo	COORDINATE MYGEST	Latitudine	Longitudine	Dec. Latitudine	Dec. Longitudine
Agosine	1	1	1_Agosine_ITVCC	4_Agosine_WiFi	TVCC	FTTH	Via XXXXX civ. XX		45°XX'XX.X"N	10°XX'XX.X"E	45.XXXXXX	10.XXXXXX
Agosine	6	1	6_Agosine_ITVCC		TVCC	FWA	Via XXXXX		45°XX'XX.X"N	10°XX'XX.X"E	45.XXXXXX	10.XXXXXX

Tabella 3.2: Tabella Excel delle coordinate geografiche(dati sensibili offuscati)

Questi due file sono depositati all’interno del file system della macchina virtuale Ubuntu Server dedicata al monitoraggio, all’interno della directory di lavoro del progetto (/home/administrator/zabbixHostsScript/). Essi vengono aggiornati periodicamente dal personale tecnico di Secoval non appena un nuovo apparato viene installato o censito sul campo.

3.3.2 L'estrazione dei dati

Purtroppo Zabbix non è in grado di leggere nativamente un file `.xlsx` per configurare i propri host. Per risolvere questo problema e automatizzare il flusso di informazioni verso Zabbix, è stato scritto uno script Python.

Il sistema opera secondo una logica basata su due fasi:

- **Estrazione:** lo script Python, sfruttando librerie dedicate alla manipolazione dati come `pandas`, accede ai percorsi assoluti dei due file Excel e ne legge le righe. Questa operazione permette di estrarre la conoscenza testuale racchiusa nelle celle (indirizzi IP, ID Palo, denominazione del Comune). I dati grezzi estratti dagli Excel vengono mappati all'interno di dizionari Python.
- **Caricamento via API:** i dati vengono inseriti all'interno del server Zabbix. Lo script Python effettua chiamate al server zabbix attraverso la libreria `py-zabbix`. Il sistema esegue un confronto tra le informazioni presenti negli Excel e quelle su Zabbix; per esempio se l'IP di un asset non è presente su Zabbix allora non viene forzata la sua creazione, se invece è presente si associano ad esso i relativi metadati (nome, ID palo, coordinate...).

3.4 Implementazione dell'aggiornamento degli host

Ogni host in Zabbix viene identificato con un attributo "name" che contiene l'IP con cui è stato scoperto. Inoltre possiede anche un attributo "visible name" che lo rappresenta nel sistema. L'obiettivo era andare a estrarre dal file Excel degli asset l'informazione del loro identificativo aziendale, da inserire nell'attributo "visible name" e l'informazione sull'id del palo su cui sono montati, da inserire nel campo "location" del loro attributo "inventory" (ogni host possiede un ampio elenco di campi da compilare nell'attributo "inventory").

Oltre a queste informazioni, per ogni host era necessario associare anche le coordinate geografiche del palo su cui sono montati.

Siccome la quasi totalità delle sottoreti al momento del mio tirocinio presentava numerosi host con svariati problemi, che impedivano al sistema di scoprirli tramite ping, è risultato impossibile assegnare i loro metadati in una sola volta. Quindi, mano a mano che essi diventano online, vengono scoperti dal sistema che provvede ad associare a essi le informazioni contenute nei file Excel. Proprio come per gli host scoperti inizialmente.

Questo implica che l'assegnazione di questi dati deve essere fatta periodicamente, con un periodo congruo. Per questo motivo si è scelto di utilizzare script python, eseguiti periodicamente dal servizio `cron` (Linux man-pages project, 2026a) presente nativamente in Ubuntu. Il periodo scelto è di un'ora, in modo da non sovraccaricare il server Ubuntu ma di rendere possibile agli operatori vedere i cambiamenti nella stessa giornata lavorativa.

Nella pratica vengono utilizzati degli script python eseguiti nella macchina virtuale che ospita Zabbix. Tutti gli script ausiliari vengono salvati nella directory `/home/administrator/zabbixHostsScripts` (con alcune eccezioni) e tutte le esecuzioni sono svolte nell'ambiente virtualizzato `venv` della stessa directory.

3.4.1 Associazione dei metadati agli host

Esistono due script python che si occupano di associare i metadati agli host, uno per l'associazione dei dati attinenti al singolo host proveniente dal file Excel degli asset e l'altro per l'associazione dei dati relativi alle coordinate geografiche del palo identificato con il proprio ID.

In entrambi i casi vengono estratti gli host dal sistema zabbix attraverso la libreria pyzabbix e le API JSON-RPC di Zabbix alla sua base. Gli host vengono confrontati con i dati presenti nei file Excel. Se ci sono incongruenze, gli host vengono aggiornati con le informazioni corrette altrimenti non avvengono modifiche agli host che posseggono già i metadati corretti.

Questi script vengono eseguiti periodicamente ogni ora tramite questi comandi crontab:

```

1 0 * * * * /home/administrator/zabbixHostsScripts/venv/bin/python3 /
  home/administrator/zabbixHostsScripts/
  zabbix_HostName_IDpalo_updater.py >> /home/administrator/
  zabbixHostsScripts/cron_log.log 2>&1
2
3 10 * * * * /home/administrator/zabbixHostsScripts/venv/bin/python3
  /home/administrator/zabbixHostsScripts/
  zabbix_palo_location_and_info_updater.py >> /home/administrator/
  zabbixHostsScripts/cron_log.log 2>&1

```

Listing 3.2: Setup dell'esecuzione periodica in Crontab

3.4.2 Memorizzazione dei messaggi di log

Gli script python visti fino a ora generano degli output testuali, che andrebbero nello standard output, ma vengono reindirizzati dai job crontab in uno specifico file di log: `cron_log.log`, ogni scrittura nel file è preceduta dal timestamp e dallo script che la genera.

Per evitare di memorizzare questi dati all'infinito e generare un file di testo gigantesco, è stato utilizzato il sistema **Logrotate** (Linux man-pages project, 2026b), una utility Linux il cui scopo è la rotazione, compressione e rimozione di file di log automatici.

In questo caso il file contenente le configurazioni per il job logrotate del file `cron_log.log` è accessibile nel file: `/etc/logrotate.d/zabbix_scripts`. Al suo interno troviamo:

```

1 #path del file di log da gestire
2 /home/administrator/zabbixHostsScripts/cron_log.log {
3     #user privileges
4     su administrator administrator
5     #periodo temporale di rotazione dei log
6     weekly
7     #quante copie di log tenere
8     rotate 12
9     #aggiungi la data al nome del file
10    dateext
11    dateformat -%Y-%m-%d
12    #dove archiviare i log
13    olddir /home/administrator/zabbixHostsScripts/storico_log
14    #comprimi i file di log vecchi ma aspetta un ciclo
15    compress
16    delaycompress

```

```

17 #ignora gli errori se il file non esiste
18 missingok
19 #non ruotare il log se e' vuoto
20 notifempty
21 # crea un nuovo file alla rotazione e assegna questi permessi
22 create 0644 administrator administrator
23 }

```

Listing 3.3: Configurazione logrotate per un file di log

3.5 Georeferenziazione e GIS integrato in Zabbix

I metadati associati agli host in Zabbix, come longitudine e latitudine, vengono utilizzati nativamente dal software GIS (geographic information system) integrato. Le coordinate geografiche degli apparati vengono estratte dai campi `location_lat` (Latitudine) e `location_lon` (Longitudine) dell'inventory degli host. All'interno del widget *GeoMap*, nativamente disponibile nelle dashboard di Zabbix grazie allo strumento OpenStreetMap (OpenStreetMap Foundation, 2026), si possono visualizzare gli host su di una cartina, codificati tramite colore per la gravità dei problemi che eventualmente riscontrano. Essi mantengono l'integrazione con il sistema di monitoraggio, quindi un host con un problema visualizzato in rosso sulla mappa permette di risalire al suo nome e caratteristiche e, se risolto, diventa verde in tempo reale.

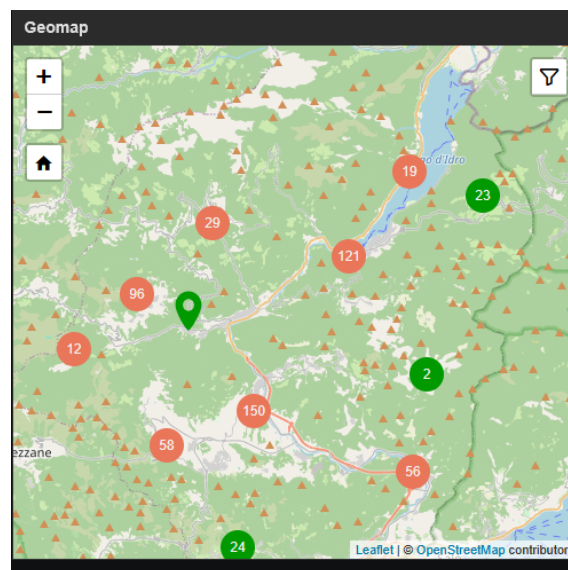


Figura 3.1: GIS integrato in Zabbix

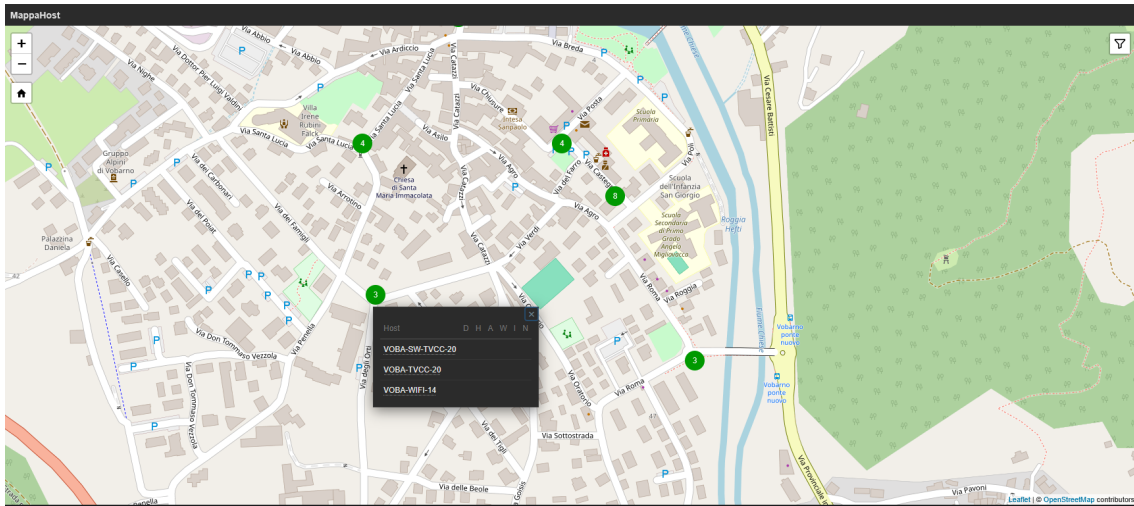


Figura 3.2: GIS integrato in Zabbix - dettaglio di un'area

3.6 Reportistica e KPI

L'implementazione di un sistema di monitoraggio centralizzato vista fino ad ora genera parallelamente dei dati. Per trasformare questa raccolta di allarmi in informazioni utili al management aziendale, è necessario estrarre e analizzare le serie temporali. Poiché Zabbix non supporta nativamente l'invio programmato di file arbitrari o analisi testuali complesse via email, è stato progettato un sistema per il salvataggio dei dati relativi agli eventi rilevati da Zabbix e un sistema per la generazione di report automatici.

3.6.1 Esportazione dati evento tramite custom AlertScript

Per ottenere l'esportazione degli eventi e dei relativi metadati degli host coinvolti, si è deciso di sfruttare il sistema di *Media Types* di Zabbix. Si è intrapresa questa strada invece di una interrogazione diretta del database MySQL di Zabbix per via dei vincoli temporali del tirocinio e per avere flessibilità di elaborazione diretta dei dati tramite le librerie Python. Solitamente, un Media Type definisce il canale di consegna di una notifica (es. Email, SMS o Telegram). Nel nostro caso, è stato creato un Media Type personalizzato di tipo **Script**.

A questo Media Type è stata agganciata un'*Action* configurata per scattare ogni volta che si verifica un problema sulla rete smart city. Invece di inviare una mail, Zabbix esegue lo script Bash `ScriptCustomExportProblemsToCSV.sh` depositato sulla macchina virtuale Ubuntu.

Zabbix passa allo script una stringa contenente tutte le **Macro** necessarie per l'analisi: i metadati relativi all'host coinvolto. Alcuni parametri in ingresso includono:

- `{EVENT.ID}`: identificativo univoco dell'allarme.
- `{EVENT.DATE}` e `{EVENT.TIME}`: timestamp di inizio del disservizio.
- `{EVENT.RECOVERY.DATE}` e `{EVENT.RECOVERY.TIME}`: timestamp di risoluzione.

- {HOST.NAME} e {INVENTORY.LOCATION}: riferimenti al dispositivo e alla sua ubicazione fisica (es. ID Palo).

3.6.2 Salvataggio locale in CSV

Lo script Bash sulla VM Linux riceve l'intera stringa di parametri e provvede ad accodarla al file CSV di destinazione.

```

1 #!/bin/bash
2
3 LOGFILE="/var/log/zabbix/ExportedProblemsToCSV.csv"
4
5 # Se il file non esiste crealo e imposta la riga di intestazione
6 if [ ! -f "$LOGFILE" ]; then
7     echo "Status,EventID,Date,Time,Host,IP,Severity,EventName,
8     TriggerName,RecoveryDate,RecoveryTime,Duration" > "$LOGFILE"
9 fi
10
11 # Appendi i dati provenienti da Zabbix
12 echo "\"$1\", \"$2\", \"$3\", \"$4\", \"$5\", \"$6\", \"$7\", \"$8\", \"$9\",
    \"$10\", \"$11\", \"$12\"" >> "$LOGFILE"

```

Listing 3.4: Custom bash script per l'esportazione in CSV

3.6.3 Elaborazione dati e report

Per estrarre significato dai dati immagazzinati che sia utile al team di manutenzione e alla direzione aziendale sono stati implementati ed eseguiti tramite **Crontab** due diversi tipi di report inviati via mail.

Report giornaliero

Il primo è un report giornaliero in cui ogni mattina vengono analizzate le entrate delle 24 ore precedenti. Le informazioni vengono divise in quattro sezioni che avranno a comporre il corpo dell'email:

- **Riepilogo:** conteggio totale dei problemi rilevati e di quelli risolti automaticamente (riconciliando le righe tramite **EventID**).
- **Problemi non risolti:** una lista degli host che hanno registrato uno stato **PROBLEM** senza un corrispondente **RESOLVED** nello stesso intervallo di tempo.
- **Infrastruttura instabile (oscillazione):** vengono contate quante volte un host ha attivato un allarme. Se il conteggio supera una soglia definita (es. 2 disconnessioni in un giorno), l'host viene evidenziato come "instabile".
- **Allarmi di sicurezza critici:** come vedremo in seguito oltre alla disconnessione il sistema può rilevare anche altri allarmi: se presenti, vengono messi in risalto per richiedere un approfondimento immediato.

Report mensile

Il secondo è un report mensile nel quale si cerca di estrapolare delle informazioni che possano evidenziare dei pattern nel modo in cui gli eventi si presentano. In modo che si possa analizzare la stabilità dell'infrastruttura e la frequenza temporale in cui si presentano le disconnessioni. Nel report sono presenti:

- **Volume degli eventi:** conteggio totale dei trigger entrati in stato **PROBLEM** e di quelli risolti (**RESOLVED**) durante l'intero mese.
- **Conteggio degli host monitorati:** il numero degli host presenti nel sistema in quel moento, in modo da tracciare la crescita dell'infrastruttura con l'aggiunta di nuovi comuni.
- **Analisi delle sottoreti:** suddivisione percentuale e numerica degli eventi problematici per tipologia di sottorete (e quindi di apparato), calcolata analizzando il terzo e quarto ottetto degli indirizzi IP (sottoreti TVCC, Switch, Access Point, connettività LTE).
- **Classifica host critici:** identificazione dei 100 dispositivi che hanno generato il maggior numero di anomalie.
- **Distribuzione temporale:** mappatura degli eventi per fascia oraria e distribuzione settimanale. Questa metrica è utile per identificare pattern di disservizio legati a momenti specifici della giornata.

Capitolo 4

Rilevamento manomissioni TVCC

Essendo le telecamere di sicurezza oggetto del progetto abbastanza moderne, è stato valutato come utile l'utilizzo dei sistemi di intelligenza artificiale implementati a bordo per rilevare diversi tipi di eventi che accadono alla singola telecamera. Tra questi troviamo la possibilità di rilevare l'oscuramento dell'obiettivo della telecamera. Per sfruttare questa funzionalità è stato creato un sistema di raccolta e invio automatico di avvisi, integrandolo con Zabbix.

4.1 Logiche di Polling vs Trapping in Zabbix

Per comprendere le scelte architetturali adottate durante il tirocinio, è utile analizzare come il server Zabbix operi per la raccolta e l'elaborazione delle metriche. Zabbix si basa sul concetto di *Item*, ovvero la singola unità di dato raccolta da un host. L'acquisizione di questi Item può avvenire seguendo due paradigmi opposti: il **Polling** (controllo attivo) e il **Trapping** (controllo asincrono o passivo).

Per la parte relativa al monitoraggio dell'accessibilità degli host alla rete e al corretto funzionamento dei servizi, è stato utilizzato il paradigma del Polling. Invece per il rilevamento delle manomissioni delle telecamere è stato utilizzato il paradigma del Trapping, in quanto le telecamere Hikvision implementano un sistema di avvisi automatici che possono essere intercettati e gestiti da Zabbix e da script ausiliari.

4.2 Descrizione dell'implementazione

Il sistema gestisce 2 eventi provenienti dalle telecamere: il tamperdetection o l'oscuramento dell'obiettivo e il motiondetection o il rilevamento di movimento. Il secondo è stato implementato solo per motivi di debug, dato che condivide gli stessi canali dell'oscuramento ma è molto più facilmente riproducibile.

Il workflow che viene eseguito per garantire la corretta gestione di queste informazioni si divide in questi step:

- Estrazione degli IP delle telecamere Hikvision presenti in Zabbix con uno script python.
- Ogni IP estratto è utilizzato per lanciare verso la telecamera le configurazioni dei loro servizi tamperdetection e motiondetection tramite configurazioni in formato XML.

- Si apre una connessione http con ogni telecamera mantenuta perpetuamente aperta in modo asincrono, attraverso uno script python. Su queste connessioni le telecamere comunicano i loro avvisi ed essi vengono comunicati a Zabbix sugli appositi Item dei trigger tramite un API CLI di Zabbix: `zabbix_sender`.
- All'interno del template relativo alle telecamere Hikvision abbiamo 2 trigger per motiondetection e oscuramento con le loro impostazioni.
- I log di questi script vengono gestiti da logrotate (Linux man-pages project, 2026b) salvandoli fino a 3 mesi.

Bottleneck: se molti allarmi si attivano contemporaneamente, lo script python lancia altrettanti script bash e ciò sovraccaricherebbe la CPU. Se l'utilizzo rimane per il tamperdetection e solo saltuariamente per motiondetection questo non è un problema.

4.3 Configurazioni tamperdetection e motiondetection hikvision tramite XML

Le telecamere Hikvision espongono una API HTTP con autenticazione che permette di modificare le configurazioni e le impostazioni della telecamera tramite l'upload di file semi strutturati XML. Essi riguardano il setup delle impostazioni e della modalità di comunicazione per 2 eventi: tamperdetection e motiondetection. Dopo avere estratto gli IP delle telecamere da Zabbix, lo script python lancia verso ogni telecamera le configurazioni necessarie per abilitare questi eventi. Esse gestiscono l'attivazione e l'impostazione dell'area dell'immagine in cui la telecamera è sensibile all'oscuramento o al motion detection e la comunicazione dell'evento triggerato all'EventManagerNotification center (lancia la comunicazione http).

4.4 Sviluppo del middleware e ascolto degli stream Hikvision ISAPI

È stato creato uno script python in `/home/administrator/zabbixHostsScripts/hikvisionOscuramentoScripts/hikvision_listener_multiplehosts.py` che si comporta come middleware. Il suo scopo principale è fare da ponte tra le telecamere di videosorveglianza Hikvision e il server di monitoraggio Zabbix, intercettando gli allarmi in tempo reale e convertendoli valori di item nel sistema Zabbix.

Invece di interrogare continuamente le telecamere (polling), lo script si mette in "ascolto" continuo sui loro flussi di eventi (streaming), catturando istantaneamente anomalie come il rilevamento del movimento (VMD) o l'oscuramento della telecamera (Tampering/Shelter alarm).

4.4.1 Implementazione del listener

Lo script si collega tramite API token a Zabbix e, utilizzando la libreria pyzabbix, estrae tutti gli host e ne salva l'indirizzo IP solo se l'host in questione è una

telecamera di marca Hikvision. Questo rende il sistema altamente scalabile: se si aggiunge una nuova telecamera su Zabbix, il riavvio successivo dello script la includerà automaticamente.

Per ogni telecamera trovata, viene avviato un "task" indipendente grazie alla libreria `asyncio`. Questo permette di gestire decine di telecamere contemporaneamente usando un solo processo leggero, senza che l'attesa di dati da una telecamera blocchi le altre. Lo script apre una connessione HTTP persistente (Stream) verso l'endpoint ISAPI della telecamera (`/ISAPI/Event/notification/alertStream`) (Hikvision Digital Technology, 2026). In questo canale, la telecamera invia frammenti in formato XML ogni volta che accade qualcosa. Lo script legge il flusso riga per riga, isola i blocchi XML significativi e li analizza in tempo reale. Estrae il tipo di evento e il suo stato (se attivo o rientrato). Quando viene rilevato un evento valido (es. movimento o telecamera coperta), lo script deve avvisare Zabbix. Per farlo lo script esegue un comando di sistema: `zabbix_sender`. Questo è uno strumento a riga di comando nativo di Zabbix, progettato specificamente per inviare dati a Zabbix. Il comando comunica a zabbix l'hostname esatto della telecamera, la chiave dell'allarme (es. `hikvision.motionDetection`) e il valore (1 per allarme attivo, 0 per allarme rientrato).

Items in Zabbix

Nel template `templateSC_configurazione_hikvision` sono presenti 2 nuovi Item di tipo *Zabbix trapper*: `AllarmeItemOscuramento(key: hikvision.oscuration)` e `AllarmeItemMotionDetection(key: hikvision.motionDetection)`. Essi sono aggiornati solo dallo script listener con `zabbix_sender`.

4.4.2 Esecuzione continuativa e log

Lo script viene configurato come servizio `systemd` (Linux man-pages project, 2026c), garantendo che si avvii automaticamente all'accensione del server. Se lo script dovesse bloccarsi per un errore critico, il sistema operativo lo riavvierà automaticamente entro 10 secondi. È stata inserita una regola che "uccide" e riavvia il servizio ogni 24 ore in modo controllato. Questo è un mezzo per forzare il sistema a ricaricare la lista aggiornata delle telecamere da Zabbix ogni giorno. Tutto l'output viene indirizzato in un file di log gestito da **Logrotate** (Linux man-pages project, 2026b) su disco per permettere il monitoraggio del corretto funzionamento.

4.4.3 Trigger actions e integrazione con Interacta

Quando un Trigger cambia stato (da OK a PROBLEM o viceversa), entra in gioco il sistema delle **Trigger Actions** native. Invece di configurare un'Action per inviare una semplice notifica e-mail, il sistema è stato configurato per eseguire un'operazione esecuzione remota di script, puntando al *custom media type: script* denominato `interactaCreateOrSolveTicket`.

La motivazione per utilizzare questo paradigma è la possibilità di iniettare dinamicamente il contesto del guasto all'interno dello script Python. L'Action è configurata per passare allo script degli argomenti tramite le Macro di Zabbix, tra cui:

- `{EVENT.ID}`: l'identificativo univoco del guasto, essenziale per correlare la futura risoluzione allo stesso ticket aperto in Interacta in cui sarà chiamato *problem_ID*.
- `{HOST.NAME}` e `{INVENTORY.LOCATION}`: per comunicare alla piattaforma l'ID dell'host e il palo fisico su cui i tecnici dovranno intervenire.
- `{TRIGGER.STATUS}`: Il valore che indica allo script se inviare alla piattaforma un payload per la creazione (PROBLEM) o per la risoluzione (RESOLVED) del ticket.

Capitolo 5

Integrazione con il sistema di ticketing Interacta

5.1 Scopo e architettura dell'integrazione

L'implementazione di un sistema di monitoraggio rappresenta solo il primo passo verso l'automazione dei processi IT aziendali. In Secoval, rilevare tempestivamente un guasto tramite Zabbix (ad esempio, l'oscuramento di una telecamera o la caduta di un Access Point) è inutile se l'informazione non raggiunge la squadra tecnica incaricata della manutenzione.

Quindi per garantire l'integrazione con il sistema già presente e il tracciamento della gestione della manutenzione e degli interventi fisici sugli apparati della Smart City, si è resa necessaria un'integrazione tra il sistema di monitoraggio Zabbix e il portale (IT Service Management) **Interacta**.

L'obiettivo dell'integrazione è automatizzare l'apertura di un ticket di assistenza (con tutti i dettagli dell'host, le coordinate e il tipo di guasto) nel momento in cui un trigger di Zabbix entra in stato **PROBLEM**, e chiuderlo in automatico quando lo stato torna a **RESOLVED**.

L'architettura si basa su uno **script Python** operante in `/usr/lib/zabbix/alertscripts/`, incaricato di elaborare i dati e comunicare con le API di Interacta. I dati necessari vengono inoltrati a esso sotto forma di arguments da un **Media Type** di tipo "Script" configurato su Zabbix. L'**autenticazione Server-to-Server** verso le API di Interacta viene eseguita tramite Service Account (Service Account Authentication è un metodo che consente ad applicazioni, script o macchine virtuali di verificare in modo sicuro la propria identità e interagire con API o servizi cloud senza intervento umano) e firma crittografica asimmetrica (RSA).

I ticket creati non devono essere generici, ma devono contenere metadati utili per i tecnici sul campo (es. l'indirizzo IP, l'ID dell'Host, l'ID del Palo fisico e la descrizione problema). Se il problema rientra autonomamente, Zabbix deve comunicare a Interacta di risolvere il ticket, evitando alle squadre tecniche uscite a vuoto.

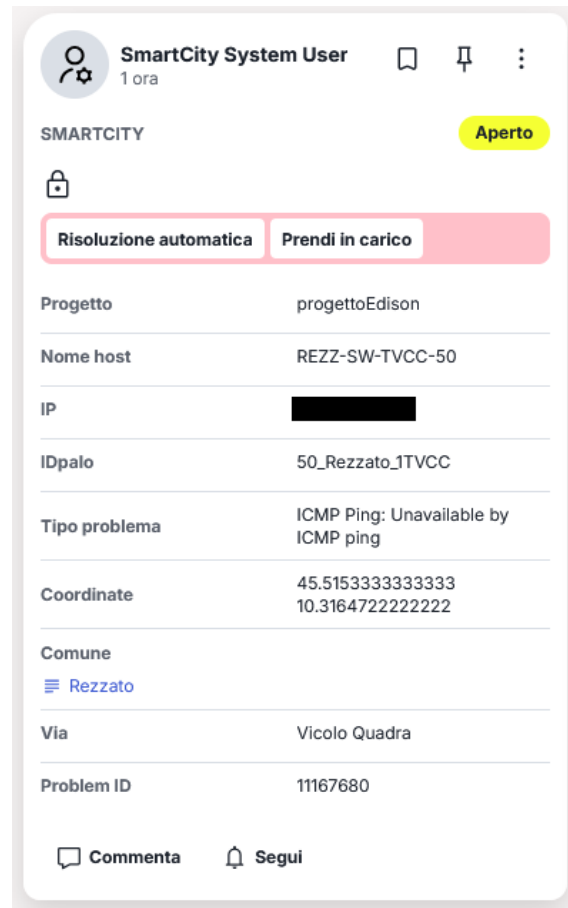


Figura 5.1: Esempio di ticket in Interacta

5.1.1 Autenticazione Server-to-Server

In un'applicazione web tradizionale, l'autenticazione si basa su interazioni umane (es. inserimento di username e password) e sul mantenimento di una sessione tramite cookie. Nel caso di questo progetto, un server Linux (Zabbix) deve dialogare autonomamente con un altro server (Interacta). Questa modalità **Server-to-Server**, richiede standard di sicurezza diversi.

La documentazione fornita da Interacta e le linee guida consigliano l'adozione di un'autenticazione basata su token crittografici, specificamente la generazione di una *JWT Assertion* (Json Web Token) (Internet Engineering Task Force (IETF), 2015) firmata con algoritmo **RS512** utilizzando una chiave privata fornita in formato JSON.

La scelta di questo framework permette di evitare che ci siano **credenziali fisse in transito** tramite la generazione di un token JWT "usa e getta" per ogni singolo ticket e singola chiamata HTTPS. Inoltre permette di garantire l'integrità, l'autenticazione e il non ripudio delle richieste HTTPS inviate dal sistema.

5.2 Generazione e firma del token JWT

5.2.1 Struttura del Token e scelta dell'algoritmo

Un JWT standard è composto da tre parti codificate in Base64URL e separate da un punto (.): l'Header, il Payload e la Signature. L'Header definisce la tipologia di token e l'algoritmo impiegato. Il Payload contiene le asserzioni e le dichiarazioni sull'entità autorizzata (es. l'identificativo del client, l'istante di emissione e il momento di scadenza). Infine, la Signature costituisce la firma digitale, che garantisce l'integrità del dato in transito.

Le specifiche di integrazione dettate da Interacta imponevano l'utilizzo dell'algoritmo **RS512** per la fase di firma. A differenza di algoritmi simmetrici, l'RS512 è asimmetrico e prevede l'uso di una coppia di chiavi: una **chiave privata**, a bordo della macchina virtuale Zabbix, utilizzata dal middleware per firmare il token in uscita, e una **chiave pubblica**, caricata in precedenza nel cloud di Interacta, utilizzata dal provider per verificare l'autenticità del token.

5.2.2 Implementazione in Python

L'implementazione del sistema di firma è stata realizzata in linguaggio Python, sfruttando la libreria `requests` per gli scambi HTTPS in sinergia con il modulo `cryptography` per la gestione sicura delle chiavi RSA.

Il payload è stato configurato, definendo un tempo di scadenza del token (`exp`) breve, solitamente impostato a 5 o 10 minuti dal momento dell'emissione (`iat`).

```

1 import uuid
2 import jwt
3 import json
4 import base64
5 import requests
6
7 from datetime import datetime
8 from cryptography.hazmat.primitives import hashes
9 from cryptography.hazmat.primitives.asymmetric import padding
10 from cryptography.hazmat.primitives.serialization import
    load_pem_private_key
11
12
13 def authentication():
14     # firma
15     try:
16         now = int(time.time())
17
18         header = {
19             "kid": key_id,
20             "alg": "RS512"
21         }
22
23         payload = {
24             "jti": str(uuid.uuid4()), # ID: Universally Unique
Identifier
25             "aud": "injenia/portal-authenticator", #audience:
destinatario del token
26             "iss": str(client_id), # issuer
27             "iat": now, # issued at: time
28             "exp": now + 600 # expiry: time

```



```

29     }
30
31     # costruzione delle prime due parti del JWT
32     b64_header = b64url_encode(json.dumps(header).encode('utf-8
33     '))
34     b64_payload = b64url_encode(json.dumps(payload).encode('utf
35     -8'))
36
37     # la stringa da firmare
38     signing_input = f"{b64_header}.{b64_payload}".encode('utf-8
39     ')
40
41     # firma Crittografica con RS512
42     private_key_obj = load_pem_private_key(private_key_pem.
43     encode('utf-8'), password=None)
44     signature = private_key_obj.sign(
45         signing_input,
46         padding.PKCS1v15(),
47         hashes.SHA512()
48     )
49
50     # Token finale
51     signed_jwt = f"{b64_header}.{b64_payload}.{b64url_encode(
52     signature)}"
53
54     # API request
55     try:
56
57         auth_url = f"{API_BASE}/core/auth/create-access-token-by-
58         service-account"
59         response = requests.post(
60             auth_url,
61             headers={"Content-Type": "application/json", "Accept":
62             "application/json"},
63             json={"jwtAssertion": signed_jwt}
64         )
65
66         response.raise_for_status()
67
68         data = response.json()
69
70         access_token = data['accessToken']
71
72         return access_token

```

Listing 5.1: Snippet Python per la generazione e firma del JWT con algoritmo RS512

5.3 Apertura o chiusura del ticket

Una volta ottenuto l'access token il passaggio successivo è utilizzarlo per autenticarsi nell'apertura o chiusura dei ticket. L'interazione tra Zabbix e lo script Python avviene tramite il passaggio di argomenti a riga di comando. In Zabbix il media type: script lanciato dalle trigger actions è configurato per iniettare dinamicamente il valore delle Macro all'interno del comando di avvio dello script.

Lo script elabora i parametri ricevuti (tra cui {EVENT.ID}, {HOST.NAME}, e {TRIGGER.STATUS}) e implementa una logica condizionale per il ciclo di vita del ticket:

- **creazione:** se il {TRIGGER.STATUS} è pari a PROBLEM, lo script compila un payload destinato all'endpoint di "richiesta", inserendo nel corpo del messaggio i dettagli dell'host, l'ID Palo e la natura del guasto. L'{EVENT.ID} generato da Zabbix viene salvato all'interno del ticket come chiave univoca di correlazione.
- **chiusura:** se il {TRIGGER.STATUS} passa a RESOLVED, lo script compila un payload destinato all'endpoint di "chiusura", inserendo nel corpo del messaggio solo l'{EVENT.ID} in modo che la piattaforma cloud di Interacta possa identificare il ticket il cui stato deve passare a "Chiuso/Risolto".

```

1 def manage_ticket(ticket_data):
2     status = ticket_data["status"]
3     event_id = ticket_data["problem_ID"]
4
5     token = authentication()
6
7     headers = {
8         "Content-Type": "application/json",
9         "Accept": "application/json",
10        "Authorization": f"Bearer {token}"
11    }
12
13    # apertura di un ticket
14    if status == "PROBLEM":
15        url = f"{API_BASE}/extensions/SmartCity/richiesta"
16
17        payload = {
18            "fields": {
19                "progetto": ticket_data["progetto"],
20                "nome_host": ticket_data["nome_host"],
21                "IP": ticket_data["IP"],
22                "IDpalo": ticket_data["ID_palo"],
23                "tipo_problema": ticket_data["tipo_problema"],
24                "coordinate": f"{ticket_data["coordinate_lat"]} {
ticket_data["coordinate_lon"]}",
25                "via": ticket_data["via"],
26                "PROBLEM ID": int(ticket_data["problem_ID"]),
27                "comune": findComuneID(ticket_data["nome_host"])
28            }
29        }
30
31        r = requests.post(url, headers=headers, json=payload,
allow_redirects=True)
32        r.raise_for_status()
33
34    # chiusura di un ticket
35    elif status == "RESOLVED":
36        url = f"{API_BASE}/extensions/SmartCity/richiesta/chiusura"
37
38        payload = {
39            "clientId": int(ticket_data["problem_ID"])
40        }
41

```

```
42         r = requests.post(url, headers=headers, json=payload,  
allow_redirects=True)  
43         r.raise_for_status()  
44  
45     else:  
46         print(f"Unknown status received: {status}. Expected PROBLEM  
or RESOLVED.")
```

Listing 5.2: Snippet Python per la apertura o chiusura di un ticket

Capitolo 6

Conclusioni

In questo capitolo vengono presentate le conclusioni del progetto, evidenziando i risultati ottenuti, il livello di soddisfacimento dei requisiti iniziali e le principali considerazioni maturate durante l'esperienza di sviluppo.

6.1 Raggiungimento degli obiettivi

Durante lo svolgimento del progetto di stage sono stati completati tutti gli obiettivi stabiliti nella fase di analisi dei requisiti. L'introduzione del sistema di monitoraggio dell'infrastruttura di rete preesistente è entrata in produzione durante gli ultimi giorni della mia permanenza in Secoval. Questo mi ha dato l'opportunità di vedere i primi frutti del mio lavoro ed anche affinare il sistema per essere più adatto all'utilizzo reale.

Obiettivo	Valutazione	Commento
Sviluppo e miglioramento del monitoraggio di rete	Raggiunto	Transizione verso un'architettura automatizzata e centralizzata basata su Zabbix 7.0, configurata per scalare facilmente in futuro.
Integrazione degli apparati con Cartografia GIS	Parzialmente raggiunto	Lo strumento di gestione e monitoraggio georeferenziato è operativo ma risiede all'interno del sistema Zabbix senza avere l'opportunità di un'integrazione con programmi più avanzati come QGIS utilizzati in azienda.
Integrazione con la piattaforma cloud ticketing aziendale Interacta	Raggiunto	Implementati script Python per gestire in sicurezza l'apertura e chiusura automatica dei ticket in modalità server-to-server.

Tabella 6.1: Riepilogo dei requisiti soddisfatti

6.2 Limiti del sistema e sviluppi futuri

Sebbene l'architettura implementata abbia raggiunto gli obiettivi prefissati, e quindi abbia ridotto il carico di lavoro manuale per la gestione dell'infrastruttura, sono consapevole che rimangano alcuni aspetti critici sia a livello implementativo che architetturale. Spesso dovuti a inesperienza e a limiti temporali.

Dal punto di vista implementativo manca un sistema di controllo di versione per il codice. Purtroppo in Secoval non è comune gestire codice e l'importanza di sistemi come git (The Git Development Community, 2026) e github non è stata oggetto di discussione. In futuro inserire il codice in una repository github potrebbe essere un buono strumento per garantire la tracciabilità e la resilienza del codice.

Inoltre non è presente un sistema di manutenzione dell'ambiente Python. Le librerie installate non sono tracciate attraverso un file standard *requirements.txt* sono indicate solamente nella documentazione da me redatta e fornita a Secoval.

Il sistema è privo di una sezione di troubleshooting: nella documentazione appena citata sono presenti i fix più comuni a cui mi sono rivolto durante l'implementazione e che pensavo potessero esser utili in futuro. Al momento non è stata fatta una analisi dei tool e delle procedure necessarie in caso di guasti al sistema.

6.3 Valutazione Personale

Il periodo di tirocinio trascorso presso Secoval ha rappresentato una importante tappa nel mio percorso di studi in Ingegneria delle Tecnologie per l'Impresa Digitale, fungendo da ponte tra l'astrazione della teoria e la concretezza dell'ambiente di produzione aziendale.

Il passaggio dallo studio universitario alla pratica in azienda mi ha posto di fronte a problematiche che non pensavo sarebbero state così preponderanti. Mentre all'università spesso si evidenzia la correttezza e gli standard dei processi, in azienda ho dovuto misurarmi con la natura imprevedibile del sistema distribuito ed eterogeneo della rete smart city gestita da Secoval.

Ho toccato con mano come l'eterogeneità di questi sistemi renda molto complesso orchestrare il monitoraggio e richieda ad ogni passaggio dei compromessi tecnici e implementativi.

Inoltre, il dialogo con il personale di Interacta, riguardo gli endpoint necessari per il corretto funzionamento del sistema di ticketing, è stato istruttivo. Ho realizzato come la creazione di sistemi, anche software, sia per la maggior parte discussione e confronto. La parte di interazione tra persone è molto importante per l'efficace realizzazione di qualsiasi progetto.

La stesura di questo elaborato, rispetto all'ambiente aziendale concentrato su logiche operative, mi ha permesso di valutare l'intero sistema in modo più critico, approfondire la documentazione e i protocolli e trasformare questa esperienza in un bagaglio di conoscenze e competenze reali.

Infine, sono stato soddisfatto di aver condotto in prima persona l'intero ciclo di vita di questo progetto. È gratificante aver rilasciato un sistema attualmente in produzione e utile nell'immediato. Questa esperienza aziendale mi lascia con una consapevolezza delle mie capacità e sicuramente meno timore nell'affrontare nuove sfide.

Glossario

Per facilitare la lettura di seguito vengono definiti i termini tecnici principali, i protocolli e i concetti architetturali utilizzati nel progetto.

Action: (Zabbix) l'azione automatica che il sistema intraprende quando qualche evento soddisfa alcune condizioni. Può essere relativa ai trigger (*Trigger Action*) o alla scoperta di un nuovo host (*Discovery Action*).

AP (Access Point): dispositivo di rete che permette la connessione wireless.

API (Application Programming Interface): un'interfaccia o "ponte" software che permette a due sistemi o applicazioni diverse (es. script Python e Zabbix) di parlarsi, passarsi comandi e scambiarsi dati in modo sicuro e automatizzato.

API JSON-RPC : (Zabbix) l'API web di Zabbix si basa integralmente sul protocollo JSON-RPC. Qualsiasi operazione eseguibile dall'interfaccia grafica (creazione host, assegnazione template, update dell'inventario) è esposta tramite questo protocollo. Es. il popolamento dell'Host Inventory tramite script Python.

Asyncio: modulo nativo di Python per la scrittura di codice concorrente e asincrono single-thread.

Base64URL: variante della codifica Base64 progettata per essere sicura all'interno degli URL. Sostituisce i caratteri potenzialmente problematici, evitando ambiguità nella trasmissione web dei token JWT.

Bash: interprete di comandi testuale predefinito nei sistemi operativi Unix e Linux, per interagire con il sistema operativo ed eseguire script.

Comunicazione asincrona: la comunicazione asincrona non richiede la presenza contemporanea degli interlocutori (es. email, ticket o messaggi su Slack e Dropbox, protocollo MQTT). Consente di gestire i messaggi senza interruzioni e di gestire il proprio tempo.

Comunicazione sincrona: la comunicazione sincrona avviene in tempo reale (es. chiamate, riunioni su Panopto o Zoom, protocollo HTTP). Le decisioni e la gestione dei messaggi devono essere rapide.

Crontab: demone di sistema nei sistemi operativi Unix-like utilizzato per pianificare l'esecuzione di comandi (Linux man-pages project, 2026a).

CSV (Comma-Separated Values): formato testuale utilizzato per l'importazione ed esportazione di dati in forma tabellare. Ogni riga rappresenta un record e i campi sono separati da un delimitatore, spesso la virgola.

- GIS (Geographic Information System):** sistemi informatici (come QGIS) progettati per acquisire, memorizzare, analizzare e gestire dati georeferenziati. Nel caso di Zabbix permettono di visualizzare gli host(apparati) e il loro stato direttamente su una mappa cartografica.
- Git / Version Control System (VCS):** sistema software progettato per tenere traccia delle modifiche apportate ai file (in particolare codice sorgente) nel tempo, permettendo di ripristinare versioni precedenti e collaborare in team (The Git Development Community, 2026).
- Host:** (Zabbix) qualsiasi dispositivo fisico o virtuale monitorato dal sistema. Nel nostro caso, una telecamera, uno switch, un Access Point o un router.
- ICMP Ping:** ICMP (Internet Control Message Protocol) è un protocollo di livello di rete utilizzato dai dispositivi per scambiare informazioni di diagnostica e notifica degli errori. L'ICMP Ping è lo strumento diagnostico principale basato su ICMP che invia una richiesta (Echo Request) e attende una risposta (Echo Reply) per verificare la connettività.
- ISAPI:** Intelligent Security API, un set di API specifico e proprietario sviluppato da Hikvision. Permette di interrogare e comandare le telecamere in modo diretto tramite protocollo HTTP e di ricevere flussi continui di avvisi di sicurezza.
- Item:** (Zabbix) la singola metrica o il singolo dato raccolto da un Host. Esempi di item sono: la risposta di un comando ping a un host o il tempo di risposta, la risposta ricevuta da un comando, il valore di un sensore fisico o virtuale.
- JWT (JSON Web Token):** standard aperto per la trasmissione sicura di informazioni tra due parti come oggetto JSON (IETF, 2026).
- KPI (Key Performance Indicator):** metriche quantificabili utilizzate per misurare l'efficacia e i progressi di un'azienda. Consentono di valutare le prestazioni e prendere decisioni basate sui dati.
- Logrotate:** utility di sistema Linux progettata per amministrare sistemi che generano un gran numero di file di log, ruotandoli e comprimendoli (Linux man-pages project, 2026b).
- LTE (Long Term Evolution):** standard di telecomunicazioni per la trasmissione dati su reti mobili. Nel progetto viene impiegato per fornire connettività ad apparati situati in aree remote.
- Machine-to-Machine:** paradigma di comunicazione in cui dispositivi hardware, sensori o server che scambiano informazioni e avviano azioni in modo autonomo, senza intervento umano.
- Macro:** (Zabbix) variabili integrate in Zabbix (scritte tra parentesi graffe, es. {HOST.NAME}) che permettono di inserire dinamicamente il nome, l'IP o altri dati all'interno di svariati campi, per esempio le mail di allarme.
- Middleware:** componente software che funge da intermediario tra applicazioni, sistemi operativi o protocolli di rete diversi, traducendo e facilitando lo scambio di dati.

- Polling:** processo di interrogazione continua e ciclica con cui un sistema verifica lo stato di dispositivi periferici. Nel caso del monitoraggio da parte di Zabbix il server centrale interroga ciclicamente e a intervalli regolari i dispositivi periferici per ottenerne lo stato operativo.
- RSA (Rivest-Shamir-Adleman):** algoritmo di crittografia asimmetrica, basato sull'utilizzo di una coppia di chiavi asimmetriche (una pubblica e una privata).
- RS512:** algoritmo di crittografia asimmetrica; RS512 utilizza RSA combinato con la funzione di hash SHA-512 per la firma digitale.
- SMTP (Simple Mail Transfer Protocol):** protocollo standard impiegato per la trasmissione di messaggi di posta elettronica tra server.
- SNMP (Simple Network Management Protocol):** il linguaggio standard universale utilizzato dai dispositivi di rete per comunicare il proprio stato di salute e la propria configurazione a un server di monitoraggio centrale.
- SSH (Secure Shell):** protocollo di rete crittografico di livello applicativo. Permette di stabilire una sessione remota tramite interfaccia a riga di comando.
- Template:** (Zabbix) un modello preconfigurato che contiene un insieme logico di Item, Trigger e regole. Viene applicato a più dispositivi per evitare di configurarli manualmente.
- Trapping:** concettualmente simile all'interrupt, dove un segnale che avvisa il nodo centrale del verificarsi di un evento prioritario, sospendendo temporaneamente la normale esecuzione. Nel caso del monitoraggio da parte di Zabbix è il dispositivo periferico (o uno script) a inviare i dati di allarme al server centrale al verificarsi di un evento.
- Trigger:** (Zabbix) una regola logica associata a un Item. Es. la telecamera non risponde al ping per 3 minuti, il Trigger "scatta" e genera un allarme (Problem).
- Troubleshooting:** processo logico in cui si ricerca, identifica e risolve un problema all'interno di un sistema informatico o altro.
- TVCC (TeleVisione a Circuito Chiuso):** sistema di telecamere per la videosorveglianza.
- Zabbix:** è una piattaforma software open-source per il monitoraggio proattivo di infrastrutture IT, reti, server, macchine virtuali e servizi cloud. Consente di raccogliere, visualizzare e analizzare metriche in tempo reale (come utilizzo CPU, RAM, rete e spazio disco), inviando avvisi immediati in caso di anomalie.
- VLAN (Virtual Local Area Network):** rete virtuale che permette di suddividere una singola rete fisica in più sottoreti logiche indipendenti. Il suo utilizzo consente di raggruppare host e dispositivi fisicamente separati sul territorio come se fossero collegati alla medesima infrastruttura di rete locale.
- VM (Virtual Machine):** macchina virtuale, ovvero l'emulazione software di un computer fisico che esegue un proprio sistema operativo in un ambiente isolato dalle risorse sottostanti.

Zabbix Sender: utility a riga di comando utilizzata per inviare dati o eventi direttamente al server Zabbix.

Bibliografia

- Canonical Ltd. (2026). *Ubuntu Server* [Ultimo accesso: 28 Giugno 2026]. <https://ubuntu.com/download/server>
- Hikvision Digital Technology. (2026). *Intelligent Security API (ISAPI) Developer Documentation* [Ultimo accesso: 8 Luglio 2026]. https://tpp.hikvision.com/download/ISAPI_OTAP
- IETF. (2026). *JSON Web Token (JWT)* [Ultimo accesso: 29 Giugno 2026]. <https://datatracker.ietf.org/doc/html/rfc7519>
- Interacta. (2026). *Interacta* [Ultimo accesso: 29 Giugno 2026]. <https://www.interacta.space/chi-siamo/>
- Internet Engineering Task Force (IETF). (2015). RFC 7519: JSON Web Token (JWT) [Ultimo accesso: 8 Luglio 2026]. <https://datatracker.ietf.org/doc/html/rfc7519>
- Linux man-pages project. (2026a). *crontab - Linux manual page* [Ultimo accesso: 8 Luglio 2026]. <https://man7.org/linux/man-pages/man5/crontab.5.html>
- Linux man-pages project. (2026b). *logrotate - Linux manual page* [Ultimo accesso: 8 Luglio 2026]. <https://man7.org/linux/man-pages/man8/logrotate.8.html>
- Linux man-pages project. (2026c). *systemd, init - Linux manual page* [Ultimo accesso: 8 Luglio 2026]. <https://man7.org/linux/man-pages/man1/init.1.html>
- OpenStreetMap Foundation. (2026). *OpenStreetMap* [Ultimo accesso: 29 Giugno 2026]. <https://welcome.openstreetmap.org/what-is-openstreetmap/>
- Paessler. (2026). *PaesslerPricingWebPage*. Paessler. Recuperato giugno 25, 2026, da <https://www.paessler.com/prtg/prtg-network-monitor#pricing>
- Python Software Foundation. (2026a). PyPI - The Python Package Index [Ultimo accesso: 8 Luglio 2026]. <https://pypi.org/>
- Python Software Foundation. (2026b). *Python Software Foundation* [Ultimo accesso: 28 Giugno 2026]. <https://www.python.org/>
- Secoval S.r.l. (2026a). *Chi Siamo*. Secoval. Recuperato giugno 19, 2026, da https://www.secoval.it/pagina21048_chi-siamo.html
- Secoval S.r.l. (2026b). *Organigramma*. Secoval. Recuperato giugno 23, 2026, da <https://trasparenza.secoval.it/archiviofile/secoval/OrganigrammaCompleto9.pdf>
- The Git Development Community. (2026). Git [Ultimo accesso: 9 Luglio 2026]. <https://git-scm.com/>
- Zabbix LLC. (2024). *Zabbix 7.0 Manual-Documentation* [Ultimo accesso: 28 Giugno 2026]. <https://www.zabbix.com/documentation/current/en/>

Ringraziamenti

Innanzitutto desidero ringraziare il mio relatore e tutor universitario Prof. Francesco Gringoli per il prezioso e costante supporto durante tutto il percorso di tirocinio e stesura della tesi. La sua competenza e disponibilità sono state fondamentali per la realizzazione di questo lavoro.

Un sincero ringraziamento va all'azienda Secoval Srl, che mi ha dato l'opportunità di svolgere il tirocinio curricolare. Un grazie particolare a Alessandro Salice, mio tutor, e al team con cui ho collaborato: lavorare con loro mi ha permesso di crescere sia professionalmente che personalmente.

Ringrazio la mia famiglia, che mi è stata di grande supporto e comprensione, rendendo possibile il completamento del mio percorso universitario.

Desidero ringraziare anche i miei amici, che pur non condividendo il percorso universitario, mi sono stati vicini con affetto e supporto.

Infine, un grazie va ai compagni universitari che nel tempo sono diventati amici, con cui ho condiviso momenti di studio e di svago.

Brescia, 15 Luglio 2026

Brando Calderara

